ViandasSemana

Módulo 3 — Pedidos

Reservas con cupo y horario de corte, cancelación y viandas a cocinar

Segundo Enriquez Gatti · Roman Oberlander
6.° 1.ª — E.T. N.° 21 — Programación sobre Redes — 2026

Módulo 3 — Pedidos

Es el corazón del negocio: el cliente reserva viandas de un menú, ve sus reservas y las cancela; quien cocina ve cuántas viandas tiene que preparar cada día.

Reservar

El cliente elige la cantidad en la tarjeta del menú y toca **"Reservar y pagar"**. El sistema controla en el servidor las reglas del emprendimiento antes de aceptar la reserva:

- **Cupo máximo:** nunca se reservan más viandas de las que la cocina puede preparar ese día, ni siquiera si dos clientes reservan en el mismo instante.
- **Horario de corte:** después de la hora de corte del menú no se aceptan reservas.
- **Rol:** sólo un CLIENTE puede reservar.

Si todo está bien, la reserva se crea **pendiente de pago**, ocupa su lugar en el cupo y el cliente pasa a pagar en Mercado Pago (módulo 4). Tiene 30 minutos para pagar, nunca más allá de la hora de corte; si no paga, la reserva vence sola y el cupo se libera.

Mis pedidos

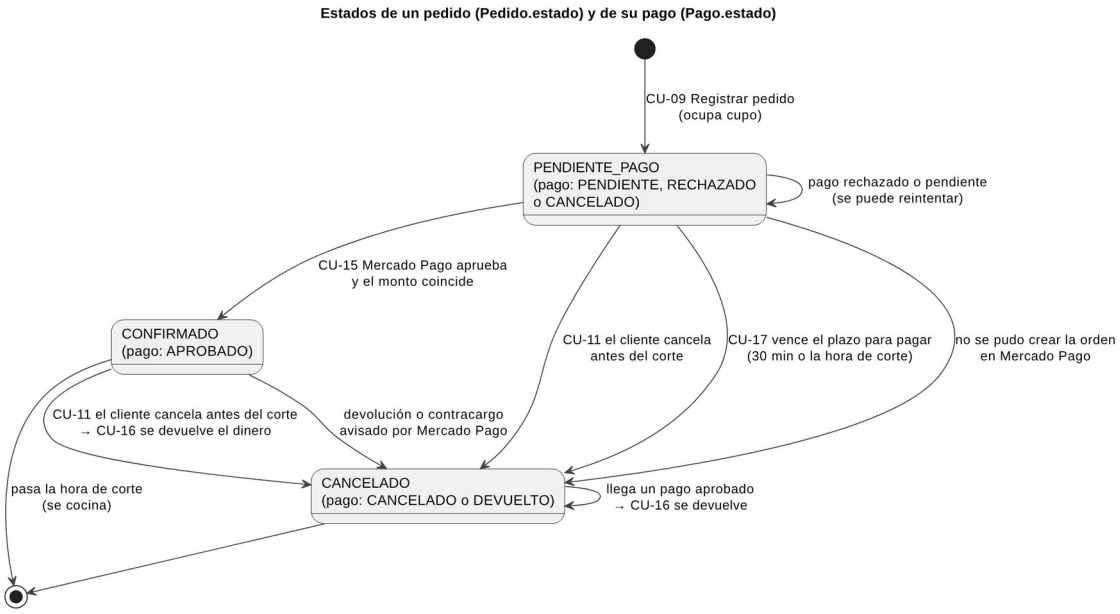
El cliente ve sus reservas agrupadas por día con el estado del pago ("Pagado", "Falta pagar", "Pago rechazado", "Cancelado") y los botones **"Pagar $..."** para reintentar y **"Cancelar"**. Es la API la que dice qué botón corresponde a cada pedido.

Cancelar

Una reserva propia se puede cancelar hasta la hora de corte. Las viandas vuelven al cupo y, si ya estaba pagada, el sistema le devuelve el dinero automáticamente por Mercado Pago. Después del corte no se puede cancelar porque la cocina ya está produciendo.

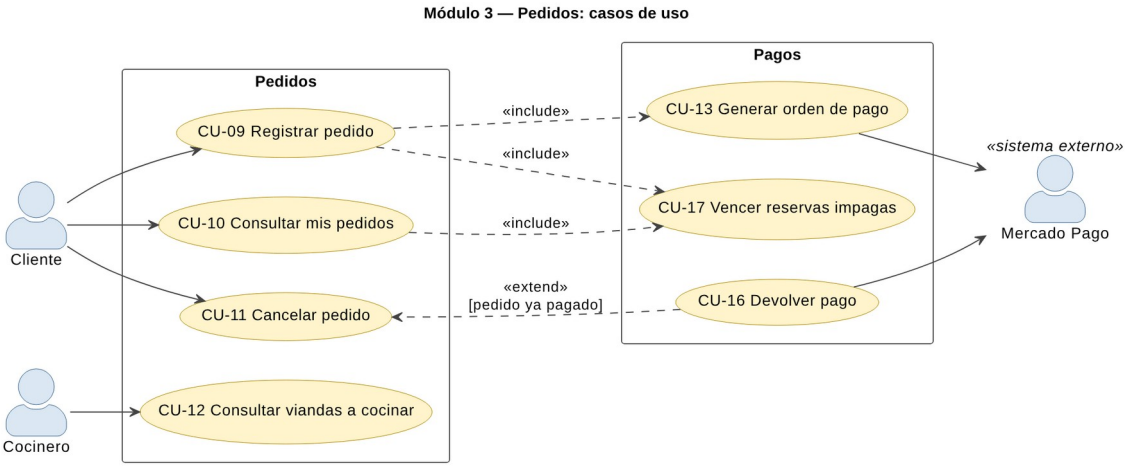
Pedidos a cocinar

El cocinero ve, por día, los pedidos **pagados** con el nombre de cada cliente y el **total de viandas a cocinar**. Las reservas que todavía no se pagaron no aparecen, porque pueden vencer.



Estados del pedido y del pago (archivo editable: diagramas/estados-pedido-pago.drawio).

Diagrama de casos de uso



Casos de uso del módulo de pedidos (archivo editable: diagramas/cu-3-pedidos.drawio).

Casos de uso

CU-09 Registrar pedido (caso de uso principal)

CU-09	Registrar pedido (caso de uso principal)		
Descripción	El cliente reserva una o más viandas de un menú. La reserva queda pendiente de pago, ocupa su lugar en el cupo y el cliente pasa a pagar en Mercado Pago.		
Actores	Cliente (principal). Mercado Pago (secundario, a través de CU-13).		
Precondiciones	El cliente tiene la sesión iniciada con rol CLIENTE. El menú existe, tiene precio y no pasó su hora de corte.		
Postcondiciones	Existe un Pedido PENDIENTE_PAGO con su Pago PENDIENTE (monto = cantidad × precio) y su URL de pago. Las viandas quedan apartadas del cupo hasta que se paguen, se cancelen o venza el plazo. Si algo falla, ninguna reserva queda ocupando cupo.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Elige la cantidad en la tarjeta de un menú y toca "Reservar y pagar".	Envía POST /api/pedidos { menuId, cantidad } con el token.
	2		Valida la sesión y que el rol sea CLIENTE.
		2.1 Si no hay sesión, el sistema deberá responder 401; si el usuario es COCINERO, 403.	
	3		Valida los datos: menuId entero positivo y cantidad entera ≥ 1.
		3.1 Si los datos son inválidos, el sistema deberá responder 400 con el detalle por campo.	
	4		Abre una transacción y bloquea la fila del menú (SELECT … FOR UPDATE): si llegan dos pedidos a la vez, el segundo espera al primero.
		4.1 Si el menú no existe, el sistema deberá responder 404 "Menú no encontrado" y deshacer la transacción.	
	5		Verifica que el menú tenga precio.
		5.1 Si el menú no tiene precio, el sistema deberá responder 409 SIN_PRECIO "Este menú todavía no tiene precio".	
	6		Cancela las reservas impagas vencidas (realiza el caso de uso CU-17) y suma las viandas de los pedidos PENDIENTE_PAGO y CONFIRMADO del menú.
	7		Verifica que no haya pasado la hora de corte y que la cantidad no supere las viandas disponibles.
		7.1 Si ya pasó la hora de corte, el sistema deberá responder 409 FUERA_DE_HORARIO y registrar el evento regla_negocio.	

		7.2 Si no alcanza el cupo, el sistema deberá responder 409 CUPO_COMPLETO ("No quedan viandas disponibles para este día" o "Sólo quedan N viandas") y registrar regla_negocio.	
	8		Crea el pedido PENDIENTE_PAGO y su pago PENDIENTE, que vence a los 30 minutos o a la hora de corte (lo que ocurra antes), y confirma la transacción.
	9		Genera la orden de pago en Mercado Pago (realiza el caso de uso CU-13) y guarda su URL.
	10		Responde 201 con el pedido, el pago y la URL de pago.
	11	Es llevado a la página de Mercado Pago.	Muestra "Te llevamos a Mercado Pago para pagar tu reserva…" y redirige (continúa en CU-14).
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	1		En el caso de que Mercado Pago no responda, devuelva un error o no esté configurado, el sistema deberá pasar el pedido y su pago a CANCELADO (libera el cupo), responder 502 o 503 con "Probá de nuevo en unos minutos" y registrar el error en el log.
	2		En el caso de que la base de datos no responda, el sistema deberá deshacer la transacción, responder 500 y registrar error_servidor.
Rendimiento	El sistema deberá realizar las acciones de los pasos 2 al 8 en un máximo de 1 segundo y el paso 9 (Mercado Pago) en un máximo de 5 segundos.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 20 veces al día.		
Importancia	Vital.		
Urgencia	Inmediatamente.		
Comentarios	Es la regla central del negocio: dos pedidos simultáneos nunca superan el cupo gracias al bloqueo de fila dentro de la transacción (hay un test que lo comprueba).		

CU-10 Consultar mis pedidos

CU-10	Consultar mis pedidos		
Descripción	El cliente ve sus reservas agrupadas por día, con el estado del pago de cada una y los botones para pagar o cancelar cuando corresponde.		
Actores	Cliente.		
Precondiciones	El cliente tiene la sesión iniciada con rol CLIENTE.		
Postcondiciones	No cambia ningún pedido, salvo las reservas vencidas, que quedan canceladas.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Abre la pestaña "Mis pedidos" (o vuelve de Mercado Pago).	Pide GET /api/pedidos con el token.
	2		Valida la sesión y cancela las reservas vencidas (realiza el caso de uso CU-17).
		2.1 Si no hay sesión, el sistema deberá responder 401 y volver al ingreso.	
	3		Lee sólo los pedidos del usuario del token, con su menú y el estado y monto de su pago, del más nuevo al más viejo.
	4		Agrega a cada pedido cancelable (no está cancelado y no pasó la hora de corte) y pagable (está PENDIENTE_PAGO).
	5	Ve sus pedidos por día.	Muestra el estado ("Pagado", "Falta pagar", "Pago rechazado", "Cancelado"...) y los botones "Pagar $ ..." y "Cancelar" según pagable y cancelable.
		5.1 Si no tiene pedidos, el sistema deberá mostrar "No tenés pedidos. Reservá desde el menú de la semana."	
		5.2 Si activa "Ver días anteriores", el sistema deberá mostrar también los pedidos de días pasados.	
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	1		En el caso de que la base de datos no responda, el sistema deberá responder 500 y registrar error_servidor.
Rendimiento	El sistema deberá realizar las acciones de los pasos 2 al 4 en un máximo de 1 segundo.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 30 veces al día.		
Importancia	Vital.		
Urgencia	Inmediatamente.		
Comentarios	El id del usuario sale del token: un cliente nunca puede ver los pedidos de otro.		

CU-11 Cancelar pedido (caso de uso principal)

CU-11	Cancelar pedido (caso de uso principal)		
Descripción	El cliente cancela una reserva propia antes de la hora de corte. Las viandas vuelven al cupo y, si la reserva ya estaba pagada, se le devuelve el dinero.		
Actores	Cliente (principal). Mercado Pago (secundario, si hay que devolver el pago).		
Precondiciones	El cliente tiene la sesión iniciada con rol CLIENTE; el pedido es suyo y no está cancelado.		
Postcondiciones	El pedido queda CANCELADO y sus viandas vuelven al cupo. Si estaba pagado, el pago queda DEVUELTO.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Toca "Cancelar" en un pedido de "Mis pedidos".	Pide confirmación; si el pedido está pagado agrega "Te devolvemos el pago por Mercado Pago".
	2	Confirma.	Envía PATCH /api/pedidos/:id/cancelar.
	3		Valida la sesión, el rol CLIENTE y que el id sea un entero positivo.
		3.1 Si el usuario es COCINERO, el sistema deberá responder 403; si el id es inválido, 400.	
	4		Abre una transacción y bloquea la fila del pedido (FOR UPDATE) para que no se confirme un pago mientras se cancela.
	5		Verifica que el pedido exista y sea del usuario del token.
		5.1 Si el pedido no existe o es de otro cliente, el sistema deberá responder 404 "Pedido no encontrado" (no revela que existe).	
	6		Verifica que el pedido no esté cancelado y que no haya pasado la hora de corte del menú.
		6.1 Si ya estaba cancelado, el sistema deberá responder 409 YA_CANCELADO.	
		6.2 Si pasó la hora de corte, el sistema deberá responder 409 FUERA_DE_HORARIO "No se puede cancelar un pedido después del horario de corte".	
	7		Si el pago estaba PENDIENTE o RECHAZADO, lo pasa a CANCELADO.
		7.1 Si el pedido estaba pagado (APROBADO), el sistema deberá realizar el caso de uso CU-16 Devolver pago; el pago queda DEVUELTO y se registra pago_devuelto.	
	8		Pasa el pedido a CANCELADO, confirma la transacción y lo devuelve.
	9	Ve el pedido cancelado.	Muestra "Pedido cancelado" y recarga la lista.

	#	Acción (actor)	Reacción (sistema)
Excepciones	1		En el caso de que Mercado Pago falle al devolver el pago, el sistema deberá deshacer la transacción, no cancelar el pedido y responder 502.
	2		En el caso de que la base de datos no responda, el sistema deberá responder 500 y registrar error_servidor.
Rendimiento	El sistema deberá realizar las acciones de los pasos 3 al 8 en un máximo de 1 segundo, o de 5 segundos si hay que devolver el pago.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 3 veces al día.		
Importancia	Vital.		
Urgencia	Inmediatamente.		
Comentarios	Es extendido por CU-16 Devolver pago con la condición [pedido ya pagado].		

CU-12 Consultar viandas a cocinar

CU-12	Consultar viandas a cocinar		
Descripción	El cocinero ve, día por día, los pedidos pagados con el nombre de cada cliente y el total de viandas que tiene que preparar.		
Actores	Cocinero.		
Precondiciones	El cocinero tiene la sesión iniciada con rol COCINERO.		
Postcondiciones	No cambia ningún dato.		
Secuencia normal	**#**	**Acción (actor)**	**Reacción (sistema)**
	1	Abre la pestaña "Pedidos a cocinar".	Pide GET /api/pedidos con el token.
	2		Valida la sesión y, por ser COCINERO, devuelve todos los pedidos con el nombre del cliente.
	3	Ve los pedidos por día.	Agrupa por día, muestra sólo los pedidos CONFIRMADO (pagados) con cantidad y cliente, y el total de viandas a cocinar de ese día.
		3.1 Si un día no tiene pedidos pagados, el sistema deberá mostrar "No hay pedidos pagados para este día."	
		3.2 Si activa "Ver días anteriores", el sistema deberá incluir los días pasados.	
Excepciones	**#**	**Acción (actor)**	**Reacción (sistema)**
	1		En el caso de que la base de datos no responda, el sistema deberá responder 500 y registrar error_servidor.
Rendimiento	El sistema deberá realizar las acciones del paso 2 en un máximo de 1 segundo.		
Frecuencia	Este caso de uso se espera que se lleve a cabo una media de 5 veces al día.		
Importancia	Vital.		
Urgencia	Inmediatamente.		
Comentarios	Es la salida que necesita la cocina para producir: cuántas viandas cocinar y para quién.		

Funcionamiento técnico

1. Arquitectura y stack tecnológico

- **API:** Express 5 + TypeScript, router `pedidosRouter` en `backend/src/routes/pedidos.ts`.
- **Reglas de negocio como funciones puras:** `validarNuevoPedido`, `validarCancelacion`, `cuposDisponibles` y `vencimientoDelPago` en `backend/src/domain/reglasPedido.ts`. No tocan la base, así que se prueban con tests unitarios.
- **Concurrencia:** transacción de Prisma (`$transaction`) con `SELECT … FOR UPDATE` en PostgreSQL.
- **Pagos:** se apoya en `lib/mercadoPago.ts` y `lib/pagos.ts` (módulo 4).
- **Frontend:** `Menus.tsx` (botón "Reservar y pagar") y `Pedidos.tsx` ("Mis pedidos" y "Pedidos a cocinar").

2. Permisos

Acción	Quién	Cómo se controla
Reservar, pagar, cancelar	CLIENTE	`exigirRol('CLIENTE')` (403 para el cocinero)
Ver pedidos	CLIENTE: sólo los suyos · COCINERO: todos	filtro `where: { usuarioId }` según el rol del token
Cancelar un pedido ajeno	nadie	`pedido.usuarioId !== req.usuario.id` → 404

3. Flujo técnico de una reserva (control de cupo)

1. zod valida `{ menuId, cantidad }`.
2. Se abre `prisma.$transaction` y se ejecuta `SELECT … FROM "MenuDia" WHERE id = $1 FOR UPDATE`: la fila del menú queda bloqueada hasta el final de la transacción.
3. Se llama a `vencerReservasImpagas()` y se suman las viandas de `OCUPAN_CUPO` (PENDIENTE_PAGO y CONFIRMADO).
4. `validarNuevoPedido({ ahora, fecha, horaCorte, cupoMaximo, reservadas, cantidad })` lanza un `HttpError(409, …, 'CUPO_COMPLETO' | 'FUERA_DE_HORARIO')` si no corresponde.
5. Se crean el `Pedido` (PENDIENTE_PAGO) y el `Pago` (PENDIENTE, `venceEn = vencimientoDelPago(...)`) y se confirma la transacción, que libera el bloqueo.
6. Fuera de la transacción se crea la preferencia en Mercado Pago. Si falla, el pedido y el pago pasan a CANCELADO.

Por qué el bloqueo: sin él, dos clientes podrían leer "quedan 2" al mismo tiempo, reservar 2 cada uno y superar el cupo. Con `FOR UPDATE` el segundo espera y lee el valor ya actualizado. El test "pedidos simultáneos nunca superan el cupo" lo comprueba.

4. Esquema en la base de datos

Tabla	Campos	Notas
Pedido	id, usuarioId → Usuario, menuId → MenuDia, cantidad, estado, creadoEn	estado: PENDIENTE_PAGO → CONFIRMADO o CANCELADO
MenuDia (lectura y bloqueo)	cupoMaximo, horaCorte, precio, fecha	el FOR UPDATE se hace sobre esta fila

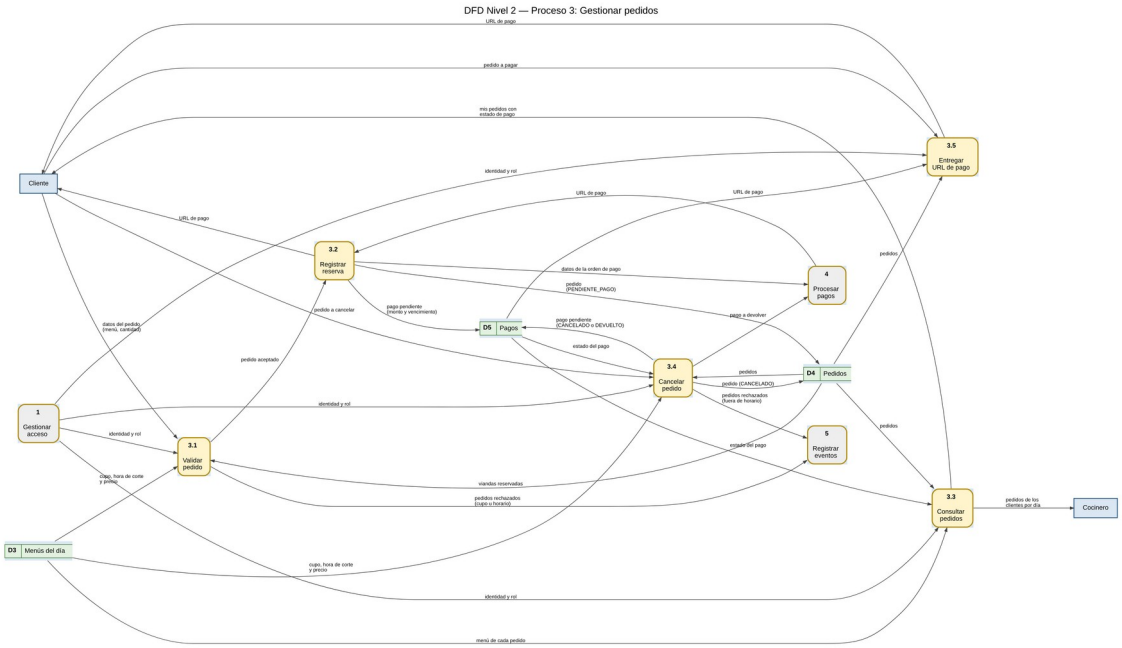
Tabla	Campos	Notas
Pago	pedidoId (único), monto, estado, venceEn, urlPago	uno por pedido (módulo 4)

5. Endpoints

Método y ruta	Rol	Respuestas
GET /api/pedidos	CLIENTE (los suyos) / COCINERO (todos)	200, 401
POST /api/pedidos	CLIENTE	201, 400, 401, 403, 404, 409, 502, 503
POST /api/pedidos/:id/pagar	CLIENTE	200 { urlPago }, 401, 403, 404, 409 (módulo 4)
PATCH /api/pedidos/:id/cancelar	CLIENTE	200, 400, 401, 403, 404, 409, 502

Códigos de negocio que acompañan al 409: `CUPO_COMPLETO`, `FUERA_DE_HORARIO`, `SIN_PRECIO`, `YA_CANCELADO`, `NO_PAGABLE`, `CANTIDAD_INVALIDA`.

6. DFD nivel 2 — proceso 3 "Gestionar pedidos"



Archivo editable: diagramas/dfd-2-pedidos.drawio.

N.º	Proceso	Entrada	Salida	Almacenes
3.1	Validar pedido	datos del pedido, identidad y rol	pedido aceptado; rechazos → 5	D3, D4
3.2	Registrar reserva	pedido aceptado; URL de pago (de 4)	datos de la orden → 4; URL → Cliente	D4, D5
3.3	Consultar pedidos	identidad y rol	mis pedidos; pedidos por día	D3, D4, D5
3.4	Cancelar pedido	pedido a cancelar, identidad y rol	pago a devolver → 4; rechazos → 5	D3, D4, D5

N.º	Proceso	Entrada	Salida	Almacenes
3.5	Entregar URL de pago	pedido a pagar, identidad y rol	URL de pago	D4, D5

7. Reglas y dónde se validan

Regla	Dónde	Por qué ahí
Formato de menuId y cantidad	`esquemaPedido` (zod)	datos de entrada → 400
Cupo máximo y hora de corte para pedir	`validarNuevoPedido`	función pura con tests unitarios
Hora de corte para cancelar	`validarCancelacion`	ídem
Pedidos simultáneos no superan el cupo	transacción + FOR UPDATE	sólo la base puede garantizarlo
Botones "Pagar" y "Cancelar"	`pagable` y `cancelable` calculados en la API	el frontend no calcula reglas

8. Archivos del código

Archivo	Contenido
backend/src/routes/pedidos.ts	Crear (transacción y bloqueo), listar, pagar y cancelar.
backend/src/domain/reglasPedido.ts	Reglas puras de cupo, corte y vencimiento.
frontend/src/pantallas/Menus.tsx	Función reservar y botón "Reservar y pagar".
frontend/src/pantallas/Pedidos.tsx	"Mis pedidos" y "Pedidos a cocinar".

9. Tests que lo cubren

- **reglasPedido.test.ts (19, unitarios):** hora de corte a UTC, un minuto antes y justo en el corte, cupo disponible nunca negativo, pedido que completa exactamente el cupo, que lo supera, cupo completo, después del corte, cantidades inválidas, cancelación antes y después del corte y ya cancelada, 30 minutos para pagar sin pasar el corte, traducción de estados de Mercado Pago.
- **pedidos.test.ts (11, integración):** pedido pendiente de pago (201), supera el cupo (409), después del corte (409), pedidos simultáneos que nunca superan el cupo, datos inválidos (400) y menú inexistente (404), cocinero que intenta pedir (403), cada cliente ve sólo lo suyo y el cocinero todo, total a preparar, cancelación que libera el cupo, doble cancelación (409) y pedido ajeno (404), cancelación después del corte (409).